MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Maxim Ormanji FAF-242

Report

Laboratory work No.5-6

Operating System

Checked by:

Reițman Petricia, *university assistant*

DISA, FCIM, UTM

Chișinău – 2026

Laboratory Work no. 5
The File System with Your Own Hands (Linux)

Purpose of the work

The purpose of this lab is to check on a real Linux system, with my own hands, the ideas of a file system: the directory tree and the file types, inodes, hard and symbolic links, permissions, and the disks under the file system. No programming is needed: only commands, careful observation and short explanations.

Environment

The lab was done on Ubuntu running in a VirtualBox virtual machine, as the normal user `vboxuser` (not root). All files of the lab are in `~/os-lab5`.

Before you start

The experiments with permissions must be done without root, because root ignores most permission checks.

```
vboxuser@Ubuntu:~$ whoami
vboxuser
vboxuser@Ubuntu:~$ mkdir -p ~/os-lab5 && cd ~/os-lab5
vboxuser@Ubuntu:~/os-lab5$
```

The command `whoami` did not print `root`, so I can continue.

Part 1. The tree and the file types

```
vboxuser@Ubuntu:~/os-lab5$ ls /
bin  boot  cdrom  dev  etc  home  lib  lib64  lost+found  media  mnt  opt  proc  root  run
sbin  snap  srv  sys  tmp  usr  var
vboxuser@Ubuntu:~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 19 11:45 /dev/null
drwxr-xr-x 142 root root 12288 Oct 19 11:45 /etc
-rw-r--r--   1 root root  2823 Oct  2 12:41 /etc/passwd
drwxrwxrwt  18 root root   420 Oct 19 11:46 /tmp
```

Figure 1.1 – The root of the tree and the types of five paths

```
vboxuser@Ubuntu:~/os-lab5$ ls -l /dev | head
total 0
crw-r--r--   1 root     root     10, 235 Oct 19 11:45 autofs
drwxr-xr-x   2 root     root        500 Oct 19 11:45 block
drwxr-xr-x   2 root     root         80 Oct 19 11:45 bsg
```

```
crw-------   1 root     root     10, 234 Oct 19 11:45 btrfs-control
drwxr-xr-x   3 root     root          60 Oct 19 11:45 bus
lrwxrwxrwx+  1 root     root           3 Oct 19 11:45 cdrom -> sr0
drwxr-xr-x   2 root     root        3740 Oct 19 11:45 char
crw-------   1 root     tty        5,   1 Oct 19 11:45 console
lrwxrwxrwx   1 root     root          11 Oct 19 11:45 core -> /proc/kcore
```

Figure 1.2 – The beginning of /dev

To find one character device and one block device I filtered the same listing by the first letter:

```
vboxuser@Ubuntu:~/os-lab5$ ls -l /dev | grep -m1 '^c'
crw-r--r--   1 root     root     10, 235 Oct 19 11:45 autofs
vboxuser@Ubuntu:~/os-lab5$ ls -l /dev | grep -m1 '^b'
brw-rw----   1 root     disk      7,   0 Oct 19 11:45 loop0
```

Figure 1.3 – A character device and a block device

```
vboxuser@Ubuntu:~/os-lab5$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
vboxuser@Ubuntu:~/os-lab5$ du -sh /var/log 2>/dev/null
112M    /var/log
vboxuser@Ubuntu:~/os-lab5$ du -sh /usr 2>/dev/null
5.3G    /usr
```

Figure 1.4 – The output of file and du

Observe (Part 1)

1. **For each path in the second line, write its type from the first letter of `ls -l`.**

Path	First letter	Type
`/etc`	`d`	directory
`/etc/passwd`	`-`	regular file
`/bin`	`l`	symbolic link
`/dev/null`	`c`	character device
`/tmp`	`d`	directory (the last letter `t` is the sticky bit, see Part 3)

2. **What is `/bin` on my system: a directory or a link, and to where?**
   It is a symbolic link (`lrwxrwxrwx`) to `usr/bin`. The command `file /bin` says the same: `symbolic link to usr/bin`. In this Ubuntu the old directories `/bin`, `/sbin` and `/lib` are only links into `/usr`. Another interesting thing: `/usr/bin/ls` is also a link, to the `coreutils` program in `/usr/lib/cargo/bin`, so this system uses the Rust version of the basic commands.

3. **One character device and one block device in `/dev`.**
   Character device: `/dev/autofs` (it starts with `c`; also `/dev/null` is one, with the numbers 1, 3). Block device: `/dev/loop0` (it starts with `b`, major number 7). A character device is read and written as a stream of bytes, a block device is accessed in blocks and can hold a file system.
4. **Which is bigger, `/var/log` or `/usr`, and by roughly how much?**
   `/usr` is bigger: 5.3G against 112M for `/var/log`. 5.3 G is about 5430 M, so `/usr` is about 48 times bigger. This is logical: `/usr` holds all the programs and libraries, and `/var/log` holds only the logs.

Part 2. Inodes and links

1. An inode, and a name that is not part of it

```
vboxuser@Ubuntu:~/os-lab5$ echo hello > notes.txt
vboxuser@Ubuntu:~/os-lab5$ stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 426610      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/vboxuser)   Gid: ( 1000/vboxuser)
Access: 2026-10-19 11:46:52.109831299 +0000
Modify: 2026-10-19 11:46:52.109831299 +0000
Change: 2026-10-19 11:46:52.109831299 +0000
 Birth: 2026-10-19 11:46:52.109831299 +0000
vboxuser@Ubuntu:~/os-lab5$ ls -li notes.txt
426610 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 19 11:46 notes.txt
```

Figure 2.1 – stat and ls -li of a new file

2. Hard link, then remove the “original”

```
vboxuser@Ubuntu:~/os-lab5$ ln notes.txt hard.txt
vboxuser@Ubuntu:~/os-lab5$ ls -li notes.txt hard.txt
426610 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 19 11:46 hard.txt
426610 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 19 11:46 notes.txt
vboxuser@Ubuntu:~/os-lab5$ rm notes.txt
vboxuser@Ubuntu:~/os-lab5$ cat hard.txt
hello
vboxuser@Ubuntu:~/os-lab5$ ls -li hard.txt
426610 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 19 11:46 hard.txt
```

Figure 2.2 – A hard link before and after removing the first name

3. Symbolic link, then remove its target

```
vboxuser@Ubuntu:~/os-lab5$ ln -s hard.txt soft.txt
vboxuser@Ubuntu:~/os-lab5$ ls -li hard.txt soft.txt
426610 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 19 11:46 hard.txt
426612 lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 19 11:46 soft.txt -> hard.txt
```

```
vboxuser@Ubuntu:~/os-lab5$ cat soft.txt
hello
vboxuser@Ubuntu:~/os-lab5$ rm hard.txt
vboxuser@Ubuntu:~/os-lab5$ cat soft.txt
cat: soft.txt: No such file or directory
vboxuser@Ubuntu:~/os-lab5$ ls -l soft.txt
lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 19 11:46 soft.txt -> hard.txt
```

Figure 2.3 – A symbolic link and a broken (dangling) link

4. Directories have links too

My prediction: a new empty directory has 2 links, and after two subdirectories it has 4.

```
vboxuser@Ubuntu:~/os-lab5$ mkdir d
vboxuser@Ubuntu:~/os-lab5$ ls -ld d
drwxrwxr-x 2 vboxuser vboxuser 4096 Oct 19 11:46 d
vboxuser@Ubuntu:~/os-lab5$ mkdir d/x d/y
vboxuser@Ubuntu:~/os-lab5$ ls -ld d
drwxrwxr-x 4 vboxuser vboxuser 4096 Oct 19 11:46 d
```

Figure 2.4 – The link count of a directory

5. Size is not space

```
vboxuser@Ubuntu:~/os-lab5$ echo x > one.txt
vboxuser@Ubuntu:~/os-lab5$ ls -l one.txt
-rw-rw-r-- 1 vboxuser vboxuser 2 Oct 19 11:46 one.txt
vboxuser@Ubuntu:~/os-lab5$ du -h one.txt
4.0K    one.txt
vboxuser@Ubuntu:~/os-lab5$ truncate -s 1G big.img
vboxuser@Ubuntu:~/os-lab5$ ls -lh big.img
-rw-rw-r-- 1 vboxuser vboxuser 1.0G Oct 19 11:46 big.img
vboxuser@Ubuntu:~/os-lab5$ du -h big.img
0       big.img
vboxuser@Ubuntu:~/os-lab5$ rm big.img
```

Figure 2.5 – The size of a file and the space it uses

Observe (Part 2)

1. **Which number in `stat` is the inode, and which tells how many names the file has?**
   The inode number is `Inode: 426610`. The number of names (hard links) is `Links: 1`. In `ls -li` the inode is the first column and the link count is the third column (the number after the permissions).
2. **After `rm notes.txt`, why could I still read `hard.txt`? What does `rm` really remove?**
   A file is not its name. The data and the information about the file (size, owner, permissions, where the blocks are) live in the inode 426610, and the names are only entries in a directory that point to this inode. After `ln` both `notes.txt` and `hard.txt` pointed to the same inode, and the link

count was 2. `rm notes.txt` removed only one name (the directory entry) and decreased the link count from 2 to 1. The inode and the data stayed, so `cat hard.txt` still printed `hello`. The data is really freed only when the link count becomes 0 (and no process has the file open).

3. **Why did `cat soft.txt` fail at the end, although `soft.txt` still exists? How big is `soft.txt` and why exactly that size?**
   A symbolic link is a separate small file with its own inode (426612, different from 426610). Inside it there is only the text of a path: `hard.txt`. When I do `cat soft.txt`, the system reads this path and looks for the file `hard.txt`. After `rm hard.txt` there is no such name, so the link is broken (dangling) and `cat` says `No such file or directory`, while the link itself is still there. Its size is 8 bytes, because it is exactly the length of the stored text `hard.txt` (8 characters).
4. **A new directory has 2 links, and 4 after two subdirectories. Where does each link come from?**
   For the empty directory `d` the two links are: (1) the name `d` in its parent directory, and (2) the entry `.` inside `d` itself, which points to `d`. After `mkdir d/x d/y` each new subdirectory has an entry `..` which points to its parent `d`. That adds two more links: $2 + 2 = 4$. So a directory has $2 +$ (number of subdirectories) links. Ordinary files inside do not change it.
5. **Why does a 2-byte file use 4.0K, and a 1 GB file use 0?**
   The file system gives space in blocks, not in bytes, and here the block is 4096 bytes (`IO Block: 4096`; in `stat` of the first file `Blocks: 8` means 8 sectors of 512 bytes = 4096). So even the file `one.txt` (the text `x` and a newline, 2 bytes) takes one whole block, and `du` shows 4.0K. The file `big.img` created with `truncate -s 1G` is a sparse file: `truncate` only sets the size 1 GB in the inode, but it does not write any data and does not allocate any block, so `du` shows 0 (the file is one big hole). `ls -l` shows the size of the file, `du` shows the space it really uses on the disk.

Part 3. Permissions

6. Files

```
vboxuser@Ubuntu:~/os-lab5$ echo secret > s.txt
vboxuser@Ubuntu:~/os-lab5$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 vboxuser vboxuser 7 Oct 19 11:47 s.txt
vboxuser@Ubuntu:~/os-lab5$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
vboxuser@Ubuntu:~/os-lab5$ chmod u+r s.txt ; cat s.txt
secret
vboxuser@Ubuntu:~/os-lab5$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0002
-rw-rw-r-- 1 vboxuser vboxuser    0 Oct 19 11:47 new.txt
drwxrwxr-x 2 vboxuser vboxuser 4096 Oct 19 11:47 newdir
```

Figure 3.1 – Permissions of files and the umask

7. Directories

My predictions: with `u=r` the command `ls box` works and `cat box/f` fails; with `u=x` the command `ls box` fails and `cat box/f` works.

```
vboxuser@Ubuntu:~/os-lab5$ mkdir box ; echo hi > box/f
vboxuser@Ubuntu:~/os-lab5$ chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied
vboxuser@Ubuntu:~/os-lab5$ chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
vboxuser@Ubuntu:~/os-lab5$ chmod 755 box
```

Figure 3.2 – Read only and execute only on a directory

8. Special bits and other people's files

```
vboxuser@Ubuntu:~/os-lab5$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 15 root root 360 Oct 19 11:46 /tmp
vboxuser@Ubuntu:~/os-lab5$ cat /etc/shadow
cat: /etc/shadow: Permission denied
vboxuser@Ubuntu:~/os-lab5$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1336 Oct  2 12:41 /etc/shadow
```

Figure 3.3 – setuid, the sticky bit and /etc/shadow

Observe (Part 3)

1. **Write 600, 644, 755 and 700 as `rwx` strings, and give one real use for each.**

Mode	String	Real use
600	`rw-------`	a private file that only the owner can read and write, e.g. a private SSH key (`~/.ssh/id_rsa`); my `s.txt`
644	`rw-r--r--`	a normal file that everybody can read but only the owner can change, e.g. `/etc/passwd`
755	`rwxr-xr-x`	a program or a public directory: everybody can run or enter it, only the owner can change it, e.g. `/usr/bin/ls`
700	`rwx------`	a private directory, e.g. `~/.ssh`

2. **With `chmod 000` I own the file but cannot read it. Why does ownership not help?**
   The system checks the permission bits of the class that matches me. I am the owner, so the "user" class is used, and for `000` this class has no `r`, so `cat` is denied. Ownership does not give access by itself; what it gives is the right to change the mode. That is why `chmod u+r s.txt` worked, and after that I could read the file again.

3. **What is my umask, and how does it explain the permissions of `new.txt` and `newdir`?**
   My umask is `0002`. The umask says which permission bits are taken away from the default ones. The default for a new file is 666, and $666 - 002 = 664$ = `rw-rw-r--` (this is what `new.txt` has). The default for a new directory is 777, and $777 - 002 = 775$ = `rwxrwxr-x` (this is what `newdir` has). So only the "write for others" bit is removed.

4. **Explain the `box` experiment: what does `r` allow on a directory, and what does `x` allow?**
   On a directory, `r` allows to read the list of names inside, and `x` allows to go through the directory, that is to access an entry by its name (to open `box/f`, to `cd` into it).
   - With `u=r`: `ls box` worked and printed `f` (the list can be read), but `cat box/f` failed, because without `x` I cannot pass through the directory to the file.
   - With `u=x`: `ls box` failed (`cannot open directory`), because I cannot read the list, but `cat box/f` worked and printed `hi`, because I know the name and I have `x`.
5. **What do the `s` in `/usr/bin/passwd` and the `t` in `/tmp` mean, and why does `passwd` need the `s`?**
   The `s` in `-rwsr-xr-x` is the setuid bit: when any user runs this program, it runs with the rights of the file owner (here `root`) and not with the rights of the user. `passwd` needs it because it must write the new password hash into `/etc/shadow`, and this file is `-rw-r-----` owned by `root:shadow`, so a normal user cannot even read it (`cat /etc/shadow` gave `Permission denied`). The `t` in `drwxrwxrwt` is the sticky bit on a directory: everybody can create files in `/tmp`, but a user can delete or rename only their own files (or the owner of the directory, or root), not the files of other users.

Part 4. Disks and the kernel as files

```
vboxuser@Ubuntu:~/os-lab5$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0     4K  1 loop /snap/bare/5
loop1    7:1    0  66.8M  1 loop /snap/core24/2124
...
loop17   7:17   0   828K  1 loop /snap/snapd-desktop-integration/391
sda      8:0    0    30G  0 disk
|-sda1   8:1    0     1M  0 part
`-sda2   8:2    0    30G  0 part /
sr0     11:0    1  50.7M  1 rom  /run/media/vboxuser/VBox_GAs_7.2.20
```

Figure 4.1 – The block devices (the 18 loop devices of the snap packages are shortened with "...")

```
vboxuser@Ubuntu:~/os-lab5$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           680M  2.1M  678M   1% /run
/dev/sda2        30G  8.1G   20G  29% /
tmpfs           1.7G     0  1.7G   0% /dev/shm
tmpfs           1.7G  8.0K  1.7G   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
tmpfs           340M   80K  340M   1% /run/user/1000
/dev/sr0         51M   51M     0 100% /run/media/vboxuser/VBox_GAs_7.2.20
vboxuser@Ubuntu:~/os-lab5$ mount | grep " / "
/dev/sda2 on / type ext4 (rw,relatime)
```

Figure 4.2 – Free space and the root file system

```
vboxuser@Ubuntu:~/os-lab5$ grep -c processor /proc/cpuinfo
2
vboxuser@Ubuntu:~/os-lab5$ head -3 /proc/meminfo
MemTotal:        3480328 kB
MemFree:          585244 kB
MemAvailable:    2084016 kB
vboxuser@Ubuntu:~/os-lab5$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   4046
Ngid:   0
vboxuser@Ubuntu:~/os-lab5$ ls -l /proc/self/fd
total 0
lrwx------ 1 vboxuser vboxuser 64 Oct 19 11:47 0 -> /dev/pts/0
l-wx------ 1 vboxuser vboxuser 64 Oct 19 11:47 1 -> pipe:[27963]
l-wx------ 1 vboxuser vboxuser 64 Oct 19 11:47 2 -> pipe:[27963]
lr-x------ 1 vboxuser vboxuser 64 Oct 19 11:47 3 -> /proc/4037/fd
```

Figure 4.3 – The kernel information in /proc

In the last command the descriptors 1 and 2 point to a pipe and not to the terminal, because I saved the output of the whole session with `tee`. In a plain terminal they would point to `/dev/pts/0`, like the descriptor 0.

Observe (Part 4)

1. **Draw my disk: disk name, partitions, which one is mounted at `/` and with what file system type.**

   ```
   sda  (30G disk)
   |-- sda1  (1M, not mounted)
   `-- sda2  (30G, mounted at /, file system type ext4)
   ```

   The only real disk is `sda` (a virtual disk of the VirtualBox machine). It has two partitions: a tiny `sda1` of 1M, which is not mounted, and `sda2` of almost the whole disk, mounted at `/`. The command `mount | grep " / "` shows that the type is `ext4`. The other devices in `lsblk` are not real disks: the `loop` devices are the images of the snap packages mounted in `/snap/...`, and `sr0` is the virtual CD with the VirtualBox Guest Additions.

2. **How full is my root file system?**
   `/dev/sda2` has 30G, 8.1G is used, 20G is free, so it is 29% full.

3. **How many CPU cores and how much RAM does my machine have, according to `/proc`?**
   `grep -c processor /proc/cpuinfo` printed 2, so there are 2 cores (virtual ones). `MemTotal` is 3480328 kB, which is $3480328/1024/1024 \approx 3.3$ GiB of RAM.

4. **`/proc/self/status` showed the name of a program. Which one, and why that one?**
   The name is `head`. `/proc/self` is a special link that always points to the directory of the process which is reading it. The process which opened `/proc/self/status` here was `head` itself, so

I saw the information about `head`. `/proc` is not a real directory on the disk: the kernel creates these "files" at the moment when a program reads them.

Conclusion (Lab 5)

In Lab 5 I checked on Linux the things that were explained in the lecture. I saw the types of files in the first letter of `ls -l` (`d`, `-`, `l`, `c`, `b`), and that `/bin` is only a link to `/usr/bin`. With `stat` and `ls -li` I saw that the name of a file is not a part of it: a file is an inode, and the names (hard links) are entries in directories, so `rm` removes a name and the data is freed only when the last name is gone. A symbolic link is a different thing: a small file that keeps a path, which becomes broken when the target is removed. I measured that the size of a file is not the space it uses (a 2-byte file takes a 4K block, a 1 GB sparse file takes nothing). I tested the permissions: the owner cannot read a file with mode 000 but can change the mode, `r` and `x` on a directory are different things (the list of names and the passing through), the umask 0002 gives 664 and 775, and the bits `s` and `t` explain how `passwd` can change `/etc/shadow` and why users cannot delete each other's files in `/tmp`. In the end I looked at the disk (`sda2`, ext4, 29% full) and at `/proc`, where the kernel shows its data as files.

Laboratory Work no. 6
Inside the xv6 File System

Purpose of the work

The purpose of this lab is to look at the ideas of a file system inside a very small teaching system, xv6. I read the xv6 file system code, write my own `stat` program (`mystat`), measure the limits written in `kernel/fs.h`, and write `find`, which walks the directory tree recursively.

Environment

The lab was done on Ubuntu (a VirtualBox virtual machine, user `vboxuser`), in the folder `~/xv6-riscv`, with the RISC-V toolchain and QEMU, started with `make qemu`. To leave QEMU I used Ctrl-A and then X.

Part 1. Read what xv6 shows you

First I looked at the type constants, then I started xv6 and ran `ls`.

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "T_DIR\|T_FILE\|T_DEVICE" kernel/stat.h
1:#define T_DIR    1 // Directory
2:#define T_FILE   2 // File
3:#define T_DEVICE 3 // Device
vboxuser@Ubuntu:~/xv6-riscv$ make qemu
...
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
args           2 3 35272
cat            2 4 36728
echo           2 5 35592
...
sync           2 26 34944
console        3 27 0
$ mkdir a
$ ls a
.              1 28 32
..             1 1 1024
```

Figure 6.1 – ls in xv6 (the middle of the list is shortened with "...")

In every line the columns are: name, type, inode number, size in bytes.

Observe (Part 1)

1. **What do types 1, 2 and 3 mean?**
   From `kernel/stat.h`: 1 is `T_DIR` (a directory), 2 is `T_FILE` (a regular file), 3 is `T_DEVICE` (a device). In the listing `.`, `..` have type 1, `README` and the programs have type 2, and `console` has type 3.
2. **Which inode number does the root directory have, and why are `.` and `..` the same there?**
   The root directory has the inode number 1. In the root both `.` and `..` show `1 1 1024`, because the root has no parent directory, so its `..` points to itself. In the directory `a` it is different: `.` is the inode 28 (the directory itself) and `..` is the inode 1 (the root).
3. **An empty directory `a` has size 32. How big is one entry, and which two entries does `a` contain?**
   A directory is a file with a table of fixed-size entries `struct dirent`: a 2-byte inode number (`ushort inum`) and a 14-byte name (`char name[DIRSIZ]`), so one entry is $2 + 14 = 16$ bytes. The size 32 is 2×16 bytes. The two entries are `.` (inode 28, the directory itself) and `..` (inode 1, the parent).

Part 2. Write mystat

The program `user/mystat.c` calls `stat()` for every argument and prints the name, the type, the inode, the number of links and the size. If `stat` fails, it writes an error to the descriptor 2 and goes to the next argument.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch(t){
  case T_DIR: return "directory";
  case T_FILE: return "file";
  case T_DEVICE: return "device";
  }
  return "?";
}

int
main(int argc, char *argv[])
{
  struct stat st;
  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
```

```
            argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }
  exit(0);
}
```

Figure 6.2 – user/mystat.c

I added the program to `UPROGS` in the `Makefile` (a TAB at the beginning, nothing after the backslash):

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "mystat\|_find" Makefile
151:    $U/_mystat\
152:    $U/_find\
```

Figure 6.3 – The new lines in UPROGS

Hard links at work:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Figure 6.4 – mystat and hard links in xv6

Two small differences from the example in the task. The `console` has the inode 27 and not 25, because I have two more programs (`mystat` and `find`) in the image. The directory `.` has 2 links and not 1, because I had created the directory `a` in Part 1 and the disk image was not rebuilt (its `..` is one more link to the root).

Then I looked at the kernel code on the host:

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "nlink--" kernel/sysfile.c
178:  ip->nlink--;
239:    dp->nlink--;
244:  ip->nlink--;
vboxuser@Ubuntu:~/xv6-riscv$ sed -n '/struct dinode/,/};/p' kernel/fs.h
struct dinode {
  short type;               // File type
  short major;              // Major device number (T_DEVICE only)
  short minor;              // Minor device number (T_DEVICE only)
  short nlink;              // Number of links to inode in file system
```

```
  uint size;               // Size of file (bytes)
  uint addrs[NDIRECT+1];   // Data block addresses
};
```

Figure 6.5 – The decrements of nlink and struct dinode

Observe (Part 2)

1. **Paste the outputs after `ln` and after `rm`. What changed and what did not?**
   The outputs are in Figure 6.4. After `ln README r2` both names show the same inode 2, the same size 2441 and `links 2`: it is one file with two names. After `rm README` only `r2` is left, and its `links` went from 2 to 1. What did not change: the inode number (2), the size (2441) and the content: `cat r2 | wc` printed `48 336 2441`, so all the 2441 bytes are still there.
2. **Find the line in `sys_unlink` that decrements the link count. When are the data blocks freed?**
   In `kernel/sysfile.c` the line is 244, `ip->nlink--;`, in the function `sys_unlink` (the line 239, `dp->nlink--;`, is in the same function and decreases the link count of the parent when a subdirectory is removed; the line 178 is in `sys_link` and only undoes the increment if `link` fails). The call to `unlink` removes the directory entry and decreases `nlink`, but it does not free the data. The blocks are freed when the link count is 0 and nobody keeps the file open: then `iput()` in `kernel/fs.c` calls `itrunc()`, which frees the data blocks, and marks the inode as free. This is the same rule that ext4 uses: a file is freed only when its last name is removed and no process keeps it open.
3. **xv6 has no symbolic links and no permissions. Which fields does an xv6 inode have?**
   `struct dinode` (the inode on the disk) has: `type` (file, directory or device), `major` and `minor` (the device numbers, only for devices), `nlink` (the number of names), `size` (in bytes) and `addrs[NDIRECT+1]` (the addresses of the data blocks: 12 direct and 1 indirect). There is no owner, no permission bits and no times, and there is no type for a symbolic link. This is much smaller than an inode of ext4.

Part 3. Measure the limits

```
vboxuser@Ubuntu:~/xv6-riscv$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define
DIRSIZ" kernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
vboxuser@Ubuntu:~/xv6-riscv$ make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

Figure 6.6 – The constants from kernel/fs.h and the disk layout

```
vboxuser@Ubuntu:~/xv6-riscv$ make qemu
...
```

```
$ mkdir averyveryverylongname
$ ls
.               1 1 1024
..              1 1 1024
README          2 2 2441
...
console         3 27 0
averyveryveryl 1 28 32
```

Figure 6.7 – A directory with a too long name

Observe (Part 3)

1. **What does `ls` show for `averyveryverylongname`, and which constant explains it?**
   `ls` shows `averyveryveryl`, the first 14 characters of the name. The constant is `DIRSIZ 14`: the name field in a directory entry (`struct dirent`) has only 14 characters, so a longer name is cut when it is written into the directory.
2. **The largest file xv6 can hold.**
   `NINDIRECT` $=$ `BSIZE`$/$`sizeof(uint)` $= 1024/4 = 256$ block numbers fit in one indirect block. An inode has `NDIRECT = 12` direct blocks and one indirect block, so the maximum is $12 + 256 = 268$ blocks.

   ```
   268 blocks * 1024 bytes = 274432 bytes = 268 KiB
   ```

3. **From the `nmeta` line: how many blocks does the whole disk have, and how many are for the log, the inodes, and the data?**
   The disk has 2000 blocks in total. Of them:
   - 1 boot block and 1 superblock;
   - 31 blocks for the log;
   - 13 blocks for the inodes;
   - 1 block for the bitmap of free blocks;
   - the first four groups together are `nmeta = 47` $(1 + 1 + 31 + 13 + 1)$ blocks;
   - 1953 blocks are for the data $(2000 - 47)$.

Part 4. Write find

The program `user/find.c` is made from `user/ls.c`. For the directory `path` it opens it, reads the `struct dirent` entries one by one, builds the full path in a buffer, prints the path if the last component is equal to `name`, and calls itself for every subdirectory. It skips the free slots (`inum == 0`) and the entries `.` and `..`, and it closes every directory it opened (a process can have only 16 open files).

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
```

```
void
find(char *path, char *name)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, 0)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }
  if(fstat(fd, &st) < 0 || st.type != T_DIR){
    close(fd);
    return;
  }
  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }
  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';
  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
      continue;
    if(strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
      continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    if(strcmp(p, name) == 0)
      printf("%s\n", buf);
    if(stat(buf, &st) < 0)
      continue;
    if(st.type == T_DIR)
      find(buf, name);
  }
  close(fd);
}

int
main(int argc, char *argv[])
{
  if(argc != 3){
    fprintf(2, "usage: find <dir> <name>\n");
    exit(1);
  }
  find(argv[1], argv[2]);
  exit(0);
```

```
}
```

Figure 6.8 – user/find.c

The test on a fresh disk (`make clean ; make qemu`):

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

Figure 6.9 – Testing find

`find . r2` found all three files named `r2`, in the order of the directory entries, and `find a b` found the directory `a/b`. Both results are the same as in the task.

One remark: the shell of xv6 reads at most 100 characters in one line. When I first typed the whole test as one long line, it was cut in the middle, so `find` started without arguments and printed `usage: find <dir> <name>`. The program was correct; I started xv6 again (the disk is kept between the runs) and ran `find a b` as a separate short line.

Conclusion (Lab 6)

In Lab 6 I found the same ideas in the small file system of xv6. The inode has only the type, the device numbers, `nlink`, the size and the block addresses; a directory is a table of 16-byte entries, which is why an empty directory is 32 bytes and a name can have only 14 characters. I wrote `mystat` and with it I saw that after `ln` two names have the same inode and `links 2`, and after `rm` the file stays with `links 1` and all its data. From `kernel/fs.h` I calculated that the largest file is $(12 + 256) \times 1024 = 274432$ bytes, and from the line `nmeta` I saw how the 2000 blocks of the disk are divided (47 blocks of metadata and 1953 blocks of data). At the end I wrote `find`, which walks the directory tree recursively with `open`, `read` of `struct dirent`, `stat` and `close`, and it found all three files `r2`. Inodes, directories as tables of names, hard links and a log are the same ideas that real file systems like ext4 use, and xv6 shows them in a few hundred readable lines.